

Lab Test 1

[25 marks]**Coding time: 1.5 hours****General Instructions:**

1. You will perform the lab test on your personal laptop. You are to disable any IDE extensions that can suggest code, e.g., Github Copilot or IntelliCode before the test. You are not allowed to use any local LLMs.
2. You are not allowed to communicate with anyone or access any network during the test.
3. You may refer to any file on your laptop during the test.
4. Make sure your code can generate **exactly the same** output as we show in the sample runs. You may be penalized for missing spaces, missing punctuation marks, misspelling, etc. in the output.
5. Do not hardcode. We will use different test cases to test and grade your solutions.
6. Follow standard Python coding conventions (e.g., naming functions and variables).
7. Python script file that cannot be executed will NOT be marked and hence you will be awarded 0 marks. You may wish to comment out the parts in your code which cause execution errors.
8. Include your name as author in the comments of all your submitted source files. For example, include the following block of comments at the beginning of each source file you need to submit.

```
# Name: Lum Ting Wong  
# Email ID: lum.ting.wong.2025
```

Instructions on how to submit your solutions:

1. Before the test begins, you will be instructed to download the zipped resource files of the test from eLearn.
2. After downloading the resource files, start Examena exam.
3. Wait for the password to unzip the .zip file.
4. After unzipping the file, **you should rename the folder to your email id**. For example, if your SMU email is lum.ting.wong.2025@scis.smu.edu.sg, you should rename the folder to **lum.ting.wong.2025**

You need to save your solutions to this folder for submission later. You may be penalized for not following our instructions.

5. When the test ends, you will be instructed to end Examena and upload your solutions as a single zip file to eLearn. For example, if your folder is named lum.ting.wong.2025, you will be uploading **lum.ting.wong.2025.zip**.

Question 1:**[Total 7 marks]****Part (a) (Difficulty Level: *)****[4 marks]**

In `q1a.py`, implement a function called `is_food_safe()`. The function takes in the following parameters:

- `temp` (type: `float`): The internal temperature of the food, in Celsius (°C).
- `holding_time` (type: `int`): The duration that the food was held at temperature `temp` in minutes.
- The function returns `True` if the food is safe to eat or `False` otherwise.

The rules for food safety are:

- If the temperature is between 75°C and 100°C (both inclusive), the food is safe.
- If the temperature is between 60°C (inclusive) and 75°C (exclusive), the food is safe only if it has been held at that temperature for at least 2 minutes.
- In all other cases, the food is unsafe.

For example:

- `is_food_safe(101.0, 2)` returns `False`
- `is_food_safe(100.0, 0)` returns `True`
- `is_food_safe(80.0, 1)` returns `True`
- `is_food_safe(74.5, 1)` returns `False`
- `is_food_safe(74.5, 2)` returns `True`
- `is_food_safe(58.0, 10)` returns `False`

Use `q1a_test.py` to test your code. Do NOT modify `q1a_test.py`. Only `q1a.py` will be graded.

Part (b) (Difficulty Level: *)**[3 marks]**

In `q1b.py`, implement a function called `compute_plan_cost()`. The function takes in the following parameters:

- `plan` (type: `str`): The type of mobile subscription plan. This string is case-sensitive. For example, 'lite' is different from 'Lite'.
- `data_used` (type: `float`): The total GB of data the user consumed in the month. You can assume that this is always a non-negative float.

The function returns the total monthly cost, rounded to 2 decimal places, for the user based on their selected plan and actual data usage. If the plan is not one of the valid options, the function **returns**: 'Invalid plan selected'.

The table below shows the monthly fees and extra data charges for the three subscription plans:

Plan	Monthly Fee	Data Quota (GB)	Extra Data Charge (per GB on pro-rated basis)
Lite	\$10	100	\$5
Standard	\$20	300	\$3
Pro	\$50	900	\$1

The monthly cost is computed as follows:

- If `data_used` is less than or equal to the plan's data quota, the total cost is the monthly fee.
- If `data_used` is greater than the plan's data quota, the total cost is the monthly fee plus the extra data charge for the amount of data used over the quota.

Examples without pro-rated extra data charges:

- `compute_plan_cost('Lite', 50.5)` returns 10
- `compute_plan_cost('Standard', 232.1)` returns 20
- `compute_plan_cost('Standard', 0.0)` returns 20
- `compute_plan_cost('Pro', 900.0)` returns 50
- `compute_plan_cost('Basic', 205.6)` returns the string 'Invalid plan selected'

Examples with pro-rated extra data charges:

- `compute_plan_cost('Lite', 100.1)` returns 10.5
- `compute_plan_cost('Standard', 333.33)` returns 119.99
- `compute_plan_cost('Pro', 999.991)` returns 149.99

Note: Make sure that the pre-defined plans, namely 'Lite', 'Standard' and 'Pro', are spelled correctly in your code. The categories are case-sensitive, e.g., 'lite' is NOT the same as 'Lite'.

You can use the `round()` function to round a float number to two decimal places. For example: `round(3.567, 2)` will return 3.57

Use `q1b_test.py` to test your code. Do NOT modify `q1b_test.py`. Only `q1b.py` will be graded.

Question 2**[Total 7 marks]****Part (a) (Difficulty Level: *)****[4 marks]**

Implement a function named `calculate_bmr` that calculates a person's Basal Metabolic Rate (BMR) in `q2a.py`. This function takes in the following parameters:

- `weight` (type: `float`): The person's weight in kilograms.
- `height` (type: `float`): The person's height in centimetres.
- `age` (type: `int`): The person's age in years.
- `gender` (type: `str`): The person's gender ('M' for male or 'F' for female).

The BMR is calculated differently for males and females:

For Men: $BMR(male) = 10 \times weight(kg) + 6.25 \times height(cm) - 5 \times age(years) + 5$

For Women: $BMR(female) = 10 \times weight(kg) + 6.25 \times height(cm) - 5 \times age(years) - 161$

The function should return the BMR (type: `int`), dropping all decimal places using the `int()` function. For example, if the calculated BMR is 1467.89, the function returns 1467.

Note: You can assume that the values for weight, height, age and gender are all valid values.

For example:

- `print(calculate_bmr(68.0, 175.0, 30, 'M'))` returns 1628
- `print(calculate_bmr(55.0, 160.0, 25, 'F'))` returns 1264

Part (b) (Difficulty Level: **)**[3 marks]**

Implement a function called `find_people_in_range` in `q2b.py`. It takes in the following parameters:

- `people (type: list[tuple])`: A list of tuples, where each tuple contains (name, weight, height, age, gender).
- `bmr_range (type: tuple)`: A tuple containing (`min_bmr`, `max_bmr`) where
 - `min_bmr` is the minimum BMR value (**inclusive**) for the desired range.
 - `max_bmr` is the maximum BMR value (**exclusive**) for the desired range.

The function returns a **string** as follows:

- The string **only** includes people whose BMR is within the specified range. The order of the people in the string must be the same as their original relative order in the input list.
- The string is formatted as a numbered, hyphen-separated list of names and BMRs: '1-name₁(bmr₁) - 2-name₂(bmr₂) ... -n-name_n(bmr_n) '

Note: You can assume valid inputs. You can **import** the `calculate_bmr` function from Part (a).

Example 1: If the list of people is:

```
people = [
    ('Apple', 45.0, 160.0, 25, 'F'),    # BMR: 1164
    ('Orange', 75.5, 178.5, 35, 'F')   # BMR: 1534
]
```

`find_people_in_range(people, (1000, 1500))` returns

1-Apple(1164)

Example 2: If the list of people is:

```
people = [
    ('Apple', 85.0, 190.0, 22, 'M'),    # BMR: 1932
    ('Orange', 55.0, 160.0, 25, 'F'),   # BMR: 1264
    ('Pear', 60.0, 165.0, 30, 'F'),     # BMR: 1320
    ('Durian', 90.0, 165.0, 45, 'M'),    # BMR: 1711
    ('Grape', 65.0, 190.0, 45, 'M'),    # BMR: 1617
]
```

`find_people_in_range(people, (1264, 1932))` returns

1-Orange(1264)-2-Pear(1320)-3-Durian(1711)-4-Grape(1617)

Question 3:**[Total 7 marks]**

Your niece loves playing the Pokémon card game and has asked you to analyze her Pokémon card collection.

Note: You cannot use dictionary for Q3. Use of dictionary will result in 0 mark.

Part (a) (Difficulty Level: **)**[3 marks]**

In `q3a.py`, define a function called `get_counts(pokemon_cards)`. This function takes in a parameter called `pokemon_cards (type: list[tuple])`, which is a list of tuples:

- Each tuple (`name`, `category`, `hp`, `attack`, `defense`) represents a Pokémon card:
 - `name (type: str)` is the name of the Pokémon, e.g., `'Pikachu'`.
 - `category (type: str)` is the type of the Pokémon. The valid categories are: `'electric'`, `'fighting'`, `'fire'`, `'ground'`, `'normal'`, or `'water'`.
 - `hp`, `attack`, and `defense (type: int)` are all integers, representing the health points, attack ability, and defense ability of the Pokémon, respectively.

The function should return a **tuple** of counts:

- Each count corresponds to the number of Pokémon of a specific category, always in the fixed order: `'electric'`, `'fighting'`, `'fire'`, `'ground'`, `'normal'`, `'water'`.

For example: If the Pokémon card collection is:

```
pokemon_cards = [('Pikachu', 'electric', 35, 55, 40),  
                 ('Charmander', 'fire', 39, 52, 43)]
```

`get_counts(pokemon_cards)` returns `(1, 0, 1, 0, 0, 0)`

Use `q3a_test.py` to test your code. Do NOT modify `q3a_test.py`. Only `q3a.py` will be graded.

Part (b) (Difficulty Level: **)**[2 marks]**

In `q3b.py`, define a function called `get_pokemon_info(pokemon_cards)`. It takes the same parameter `pokemon_cards` as in part (a).

The function should return a tuple that consists of the following statistics in order:

- `average_hp (type: float)`: rounded to 2 decimal places using the `round()` function.
- `strongest_attacker (type: str)`: name of the Pokémon with the maximum attack.
- `strongest_defender (type: str)`: name of the Pokémon with the maximum defense.

Note: You can assume that each Pokémon has a unique name, and that **no two Pokémon have the same attack or defense ability**.

For example: If the Pokémon card collection is:

```
pokemon_cards = [('Pikachu', 'electric', 35, 55, 40),  
                 ('Charmander', 'fire', 39, 52, 43)]
```

`get_pokemon_info(pokemon_cards)` returns `(37.0, 'Pikachu', 'Charmander')`

Use `q3b_test.py` to test your code. Do NOT modify `q3b_test.py`. Only `q3b.py` will be graded.

Part (c) (Difficulty Level: **/*)****[2 marks]**

In `q3c.py`, define a function called `get_best_avg(pokemon_cards, metric)`, which takes two parameters:

- `pokemon_cards` is the same as in part (a) and (b).
- `metric` (type: `str`) is one of the three strings: `'hp'`, `'attack'`, or `'defense'`.

The function determines, for the Pokémon in `pokemon_cards`, which category (`'electric'`, `'fighting'`, `'fire'`, `'ground'`, `'normal'`, or `'water'`) has the highest average on the metric. The function should return a tuple with the following:

- `best_category` (type: `str`): category (`'electric'`, `'fighting'`, `'fire'`, `'ground'`, `'normal'`, or `'water'`)
- `average_on_metric` (type: `float`): the average of that metric for the Pokémon in that category, rounded to 2 decimal places with the `round()` function.

Note: You can assume that (1) there will be no ties on the average; (2) every category has at least one Pokémon in `pokemon_cards`; and (3) `metric` is case-sensitive and will always be `'hp'`, `'attack'`, or `'defense'`.

For example: If the Pokémon card collection is:

```
pokemon_cards = [('Arcanine', 'fire', 91, 110, 80),
                  ('Pikachu', 'electric', 35, 55, 40),
                  ('Machop', 'fighting', 70, 80, 50),
                  ('Charmander', 'fire', 69, 52, 43),
                  ('Charizard', 'fire', 78, 84, 78),
                  ('Diglett', 'ground', 10, 56, 25),
                  ('Eevee', 'normal', 55, 55, 52),
                  ('Squirtle', 'water', 44, 48, 68),
                  ('Raichu', 'electric', 60, 90, 55)]
```

`get_best_avg(pokemon_cards, 'hp')` returns `('fire', 79.33)` because the three Pokémon of the fire category has the best average value of 79.33.

category	average
electric	$(35 + 60)/2 = 47.5$
fighting	$70/1 = 70.0$
fire	$(91 + 69 + 78)/3 = 79.33$
ground	$10/1 = 10.0$
normal	$55/1 = 55.0$
water	$44/1 = 44.0$

`get_best_avg(pokemon_cards, 'defense')` returns `('water', 68.0)` because the one Pokémon of the water category has the best average value of 68.0.

category	average
electric	$(40 + 55)/2 = 47.5$
fighting	$50/1 = 50.0$
fire	$(80 + 43 + 78)/3 = 67.0$
ground	$25/1 = 25.0$
normal	$52/1 = 52.0$
water	$68/1 = 68.0$

Use `q3c_test.py` to test your code. Do NOT modify `q3c_test.py`. Only `q3c.py` will be graded.

Question 4:**[Total 4 marks]****Part (a) (Difficulty Level: ***)****[2.5 marks]**

Company A has N employees seated in a row of adjacent cubicles ($N > 0$). Each cubicle is occupied by exactly one employee. Two employees are neighbors if their cubicles are next to each other (left/right). For example, in a company with 4 employees shown below, Employee 1 has one neighbour: Employee 2; Employee 2 has two neighbors: Employee 1 and Employee 3.

Employee 1	Employee 2	Employee 3	Employee 4
------------	------------	------------	------------

In `q4a.py`, implement a function called `get_min_allocation()` that takes one parameter:

- `rating` (type: `list[int]`): a **non-empty** list of N integers (≥ 0), representing employee ratings in left-to-right seating order. For example, `rating = [0, 3, 3, 5]` means the four employees (from left to right) have ratings of 0, 3, 3, and 5, respectively.

The function returns a list of N integers (type: `list[int]`) giving the allocated bonus points to employees in the same seating order. For example, a returned list of `[1, 2, 1, 2]` means that the four employees (from left to right) receive 1, 2, 1, and 2 bonus points, respectively as follows.

	Employee 1	Employee 2	Employee 3	Employee 4
Rating	0	3	3	5
Bonus Point	1	2	1	2

You must assign **integer** bonus points using the following rules:

- Rule 1:** Each employee gets **at least 1** bonus point.
- Rule 2:** If an employee's rating is **strictly higher** than a neighbor's score, then that employee must receive **strictly more** bonus points than that neighbor.
Note: No constraint for bonus points between non-neighbor ratings or equal neighbor ratings.
- Rule 3:** Because of budget cuts, the total bonus points across all employees should be **minimized**.

For example, the function should return `[1, 2, 1, 2]` in the earlier example:

- Rule 1 is fulfilled because every element in the list `[1, 2, 1, 2]` is at least 1.
- Rule 2 is fulfilled because for the following pairs of neighbors:
 - Employee 1 and 2: As Employee 2 has higher rating than Employee 1 ($3 > 0$), Employee 2 should have more bonus points than Employee 1.
 - Employee 2 and 3: Equal ratings do not impose any constraint on their bonus points.
 - Employee 3 and 4: As Employee 4 has higher rating than Employee 3, Employee 4 should have more bonus points than Employee 3.
- Rule 3 is satisfied because `[1, 2, 1, 2]` uses the minimum total bonus points. Other combinations of bonus points such as `[1, 2, 2, 3]` or `[2, 3, 1, 2]` fulfill Rule 1 and 2 but use more points.

Other examples:

- `get_min_allocation([3])` returns `[1]`
- `get_min_allocation([0, 3, 5])` returns `[1, 2, 3]`
- `get_min_allocation([9, 6, 5, 2])` returns `[4, 3, 2, 1]`
- `get_min_allocation([0, 3, 3, 5])` returns `[1, 2, 1, 2]`
- `get_min_allocation([7, 6, 6, 3])` returns `[2, 1, 2, 1]`
- `get_min_allocation([2, 6, 5, 6, 2])` returns `[1, 2, 1, 2, 1]`

Use `q4a_test.py` to test your code. Do NOT modify `q4a_test.py`. Only `q4a.py` will be graded.

Part (b) (Difficulty Level: *)****[1.5 marks]**

Company B has a total of $M \times N$ employees seated in an office laid out as a M by N grid (M rows, each with N cells). Each cell represents one cubicle and is occupied by exactly one employee. Two employees are neighbors if their cubicles share an edge (**up, down, left, right**). For example, in the 3×4 office grid below, Employee A has two neighbours: B and E; Employee B has three neighbors: A, C, and F; Employee F has four neighbours: B, J, E, and G.

A	B	C	D
E	F	G	H
I	J	K	L

In `q4b.py`, implement a function called `get_min_allocation_grid()` that takes one parameter:

- `rating_grid` (type: `list[list[int]]`): a **non-empty** list of lists representing employee ratings in seating order within the $M \times N$ office grid. For example, for a 3×4 office grid,

```
rating_grid = [[1, 2, 3, 1],
               [0, 1, 5, 2],
               [2, 3, 4, 1]]
```

means the following ratings for employees according to their seating:

1	2	3	1
0	1	5	2
2	3	4	1

The function returns a list of lists (type: `list[list[int]]`) giving the bonus points to employees in their seating order within the $M \times N$ office grid.

Just as in part (a), you must assign **integer** bonus points using the following rules:

- Rule 1:** Each employee gets **at least 1** bonus point.
- Rule 2:** If an employee's rating is **strictly higher** than a neighbor's rating, then that employee must receive **strictly more** bonus points than that neighbor. (**Note:** No constraint for bonus points between equal neighbor ratings.)
- Rule 3:** Because of budget cuts, the total bonus points across all employees should be **minimized**.

Example 1: When `rating_grid = [[1, 0, 2], [0, 1, 2]]`

`get_min_allocation_grid(rating_grid)` returns `[[2, 1, 2], [1, 2, 3]]`

This is because the minimum bonus points according to seating grid and ratings should be:

2	1	2
1	2	3

Example 2: When `rating_grid = [[1, 2, 3, 1], [0, 1, 5, 2], [2, 3, 4, 1]]`

Lab Test 1 – COR-IS1704

```
get_min_allocation_grid(rating_grid) returns [[2, 3, 4, 1],
                                              [1, 2, 5, 2],
                                              [2, 3, 4, 1]]
```

This is because the minimum bonus points according to seating grid and ratings should be:

2	3	4	1
1	2	5	2
2	3	4	1

Example 3: When `rating_grid = [[1, 2, 3, 1, 7],`
`[0, 1, 7, 2, 1],`
`[2, 3, 4, 1, 5]]`

```
get_min_allocation_grid(rating_grid) returns [[2, 3, 4, 1, 2],
                                              [1, 2, 5, 2, 1],
                                              [2, 3, 4, 1, 2]]
```

This is because the minimum bonus points according to seating grid and ratings should be:

2	3	4	1	2
1	2	5	2	1
2	3	4	1	2

Example 4: When `rating_grid = [[3, 2, 1, 1, 7],`
`[0, 1, 7, 2, 2],`
`[2, 4, 4, 1, 5],`
`[6, 5, 4, 4, 5]]`

```
get_min_allocation_grid(rating_grid) returns [[4, 3, 1, 1, 2],
                                              [1, 2, 3, 2, 1],
                                              [2, 3, 2, 1, 2],
                                              [5, 4, 1, 2, 3]]
```

This is because the minimum bonus points according to seating grid and ratings should be:

4	3	1	1	2
1	2	3	2	1
2	3	2	1	2
5	4	1	2	3

Use `q4b_test.py` to test your code. Do NOT modify `q4b_test.py`. Only `q4b.py` will be graded.

*** END OF PAPER ***

Lab Test 1 – COR-IS1704
